

3-ma'ruza. Vorislik

Reja:

1. Vorislik asoslari
2. Sif a'zolariga kirish va vorislik
3. Konstruktorlar va vorislik
4. Tayanch sinf konstruktorlarini chaqirish

Vorislik - obyektga yo'naltirilgan dasturlashning asosiy tamoyillaridan biri bo'lib, u ierarxik jarayonlarni yaratishga imkon beradi. Vorislik orqali ko'plab bog'liq elementlar uchun umumiy bo'lgan xossalarni belgilaydigan umumiy sinf yaratish mumkin. Sinflar keyinchalik bu sinfdan voris bo'lib, unga o'zlarining alohida xossalarini qo'shishi mumkin.

C# da voris oladigan sinf tayanch sinf, voris olgan sinf esa hosilaviy sinf deb ham ataladi. U tayanch sinfda belgilangan barcha o'zgaruvchilar, metodlar, xossalar va indeksatorlarni voris qilib oladi va ularga o'z a'zolarini qo'shadi.

Vorislik asoslari

C# da vorislik shundan iboratki, bitta sinf e'loniga boshqa sinfni kiritish mumkin. Buning uchun voris sinfni e'lon qilishda tayanch (asosiy) sinf ko'rsatiladi. Quyida kvadrat, to'rtburchak, uchburchak va kabi tekislikdagi obyektning asosi va balandligini o'z ichiga olgan Tekislik sinfi mavjud.

```
// Ikki o'lchovli obyektlar uchun Tekislik sinfi
```

```
class Tekislik
{
    public double Asos;
    public double Balandlik;
    public void Show()
    {
        Console.WriteLine("Asos va Balandlik " +
            Asos + " va " + Balandlik);
    }
}
```

Tekislik sinfini tayanch sinf sifatida olish mumkin, ya'ni ikki o'lchovli obyektlarning muayyan turlarini tavsiflovchi sinflarni yaratish uchun boshlang'ich nuqta. Masalan, quyidagi dasturda Tekislik sinfi Uchburchakdan voris olish uchun ishlatiladi.

Uchburchak sinfi Tekislik sinfiga voris sinf hisoblanadi

```
using System;
```

```
// Ikki o'lchovli obyektlar uchun Tekislik sinfi
```

```
class Tekislik
{
    public double Asos;
    public double Balandlik;
    public void Show()
    {
        Console.WriteLine("Asos va balandlik: " +
            Asos + " va " + Balandlik);
    }
}

// Tekislik sinfidan voris olingan Uchburchak sinfi.
class Uchburchak : Tekislik
{
    public string UchburchakTuri; // Uchburchak turi
    // Uchburchak yuzasini hisoblash
    public double Yuza()
    {
```

```

        return Asos * Balandlik / 2;
    }
    // Uchburchak turini chiqarish.
    public void ShowUchburchakTuri()
    {
        Console.WriteLine("Uchburchak: " + UchburchakTuri);
    }
}
class Shapes
{
    static void Main()
    {
        Uchburchak t1 = new Uchburchak();
        Uchburchak t2 = new Uchburchak();
        t1.Asos = 4.0;
        t1.Balandlik = 4.0;
        t1.UchburchakTuri = "teng yonli";
        t2.Asos = 8.0;
        t2.Balandlik = 12.0;
        t2.UchburchakTuri = "to'g'ri burchakli";
        Console.WriteLine("t1 obyekti bo'yicha ma'lumot: ");
        t1.ShowUchburchakTuri();
        t1.Show();
        Console.WriteLine("Yuza: " + t1.Yuza());
        Console.WriteLine();
        Console.WriteLine("t2 obyekti bo'yicha ma'lumot: ");
        t2.ShowUchburchakTuri();
        t2.Show();
        Console.WriteLine("Yuza: " + t2.Yuza());
        Console.ReadKey();
    }
}

```

Ushbu dastur bajarilganda quyidagi natija olinadi:

t1 obyekti bo'yicha ma'lumot:

Uchburchak: teng yonli

Asos va balandlik: 4 va 4

Yuza: 8

t2 obyekti bo'yicha ma'lumot:

Uchburchak: to'g'ri burchakli

Asos va balandlik: 8 va 12

Yuza: 48

Uchburchak sinfi Tekislik sinfiga tegishli obyektning maxsus turini yaratadi. Bundan tashqari, Uchburchak sinfiga Tekislik sinfining barcha a'zolari kiradi, xususan, Yuza() va ShowUchburchakTuri() metodlari qo'shiladi. Shunday qilib, uchburchak turi tavsifi UchburchakTuri o'zgaruvchisida saqlanadi, Yuza() metodi uchburchak yuzasini hisoblab chiqadi va qaytaradi, ShowUchburchakTuri() metodi esa uchburchak turini ko'rsatadi. Tekislik sinfidan voris olish uchun Uchburchak sinfi sintaksisi quyidagicha bo'ladi.

```

class Uchburchak : Tekislik
{
    //Sinf tanasi
}

```

Ushbu sintaksisni umumlashtirish mumkin. Har doim bir sinf boshqasidan voris olganda, voris sinf nomidan keyin ikki nuqta (:) va tayanch sinf nomi qo'yiladi. C# da sinf vorisi sintaksisi juda sodda va ulardan foydalanish oson.

Uchburchak sinfi o'zining Tekislik tayanch sinfining barcha a'zolarini o'z ichiga oladi, shuning uchun Asos va Balandlik o'zgaruvchilari Yuza() metodi uchun mavjud. Bundan tashqari, Main() metodidagi t1 va t2 obyektlari Asos va Balandlik o'zgaruvchilariga xuddi Uchburchak sinfi a'zolari kabi bevosita kirish mumkin.



6.5.1-rasm. Uchburchak sinfining sxematik tasviri

Tekislik sinfi Uchburchak sinfi uchun tayanch sinf bo'lishiga qaramay, mustaqil sinf hisoblanadi. Agar sinf voris olingan sinf uchun asos bo'lib xizmat qilsa, uni mustaqil ishlatib bo'lmaydi degani emas. Masalan, quyidagi dastur kodini ham ishlatish mumkin.

```
Tekislik shakl = new Tekislik();
```

```
    shakl.Asos = 10;
```

```
    shakl.Balandlik = 20;
```

```
    shakl.ShowDim();
```

Tekislik sinfi obyekti Tekislik sinfidan olingan voris sinflarning hech biriga bog'lanmagan va ularga umuman kirish huquqiga ega emas.

Quyida tayanch sinfdan voris bo'lgan sinfni e'lon qilishning umumiy shakli keltirilgan.

```
class Voris_sinf : Tayanch_sinf
```

```
{
```

```
    // Sinf tanasi
```

```
}
```

Har qanday voris sinf uchun faqat bitta tayanch sinf ko'rsatilishi mumkin. C# da bitta voris olingan sinfda bir nechta tayanch sinflarni voris qilib olish ta'minlanmagan. (Shu jihatdan C# tili bir nechta tayanch sinflarni voris qilib olish imkonini beruvchi C++ tilidan farq qiladi) Lekin vorislik iyerarxiyasini yaratish mumkin, Bu yerda, voris olingan sinf boshqa bir voris olingan sinf uchun tayanch sinfga aylanadi. Har qanday holatda voris sinf tayanch sinfning barcha a'zolarini, jumladan, o'zgaruvchilar, metodlar, xossalar va indeksatorlarini voris qilib olishi mumkin.

Vorislikning asosiy afzalligi shundaki, ko'pgina obyektlar uchun umumiy atributlarni belgilaydigan tayanch sinf yaratilgandan so'ng undan istalgancha voris sinflarda foydalanish mumkin va har bir voris olingan sinf o'z a'zosini qurishi mumkin. Masalan, quyida Tekislik sinfidan voris olingan Kvadrat sinfi keltirilgan.

```
// Tekislik sinfidan voris olingan Turtburchak sinfi
```

```
class Turtburchak : Tekislik
```

```
{
```

```
    // Agar to'rtburchak kvadrat bo'lsa, true qiymatni qaytaradi
```

```
public bool Kvadrat()
```

```
{
```

```
    if (Asos == Balandlik) return true;
```

```
    return false;
```

```
}
```

```
    // To'rtburchakning yuzasini qaytarish
```

```
public double Yuza()
```

```
{
```

```
    return Asos * Balandlik;
```

```
}  
}
```

Turtburchak sinfi Tekislik sinfining barcha a'zolarini o'z ichiga oladi, unga to'rtburchakning kvadrat ekanligini aniqlaydigan Kvadrat() metodi va to'rtburchakning yuzasini hisoblaydigan Yuza() metodi qo'shiladi.

Sinf a'zolariga kirish va vorislik

Sinf a'zolari ko'pincha ruxsatsiz kirishning oldini olish uchun yopiq deb e'lon qilinadi. Ammo sinf vorisi sinfning yopiq a'zolariga kirish cheklovlarini olib tashlamaydi. Shuning uchun agar voris olingan sinf tayanch sinfning barcha a'zolarini o'z ichiga olsa, tayanch sinfning yopiq bo'lgan a'zolariga ham kirolmaydi. Masalan, Tekislik sinfining o'zgaruvchilarini yopiq qilib e'lon qilsangiz, ularga Uchburchak sinfida kirib bo'lmaydi.

```
// Yopiq sinf a'zolari voris qilib olinmaydi  
// Ushbu kod kompilyatsiya qilinmaydi  
using System;  
class Tekislik  
{  
    double Asos; // yopiq o'zgaruvchi  
    double Balandlik; // yopiq o'zgaruvchi  
    public void Show()  
    {  
        Console.WriteLine("Asos va Balandlik: " +  
            Asos + " va " + Balandlik);  
    }  
}  
// Tekislik sinfidan voris olingan Uchburchak sinfi  
class Uchburchak : Tekislik  
{  
    public string UchburchakTuri; // uchburchak turi  
        // Uchburchak yuzasini hisoblash  
    public double Yuza()  
    {  
        return Asos * Balandlik / 2; // Xatolik beradi, chunki yopiq a'zo  
    }  
    // Uchburchak turini chiqarish  
    public void ShowUchburchakTuri()  
    {  
        Console.WriteLine("Uchburchak: " + UchburchakTuri);  
    }  
}
```

Uchburchak sinfi kompilyatsiya qilinmaydi, chunki Asos va Balandlik o'zgaruvchilariga Yuza() metodidan kirishga ruxsat berilmaydi. Asos va Balandlik o'zgaruvchilari yopiq bo'lganligi uchun faqat Tekislik sinfining boshqa a'zolari uchun ochiq, lekin voris olingan sinflar uchun yopiq.

Eslatma: yopiq sinf a'zosi o'z sinfida ochiq bo'ladi. Unga sinfdan tashqaridagi kodlardan, jumladan, voris olingan sinflardan kirish mumkin emas.

C# da yopiq a'zolariga kirishning turli usullari mavjud. Ulardan biri himoyalangan (protected) sinf a'zolaridan foydalanish, ikkinchisi esa yopiq a'zolariga kirish uchun ochiq xossalardan foydalanishdir.

Xossalari o'zgaruvchiga kirishni boshqarish imkonini beradi. Masalan, xossadan foydalanib, o'zgaruvchining qiymatiga kirish uchun cheklov qo'yish yoki uni faqat o'qish uchun e'lon qilish mumkin. Masalan, agar xossani ochiq qilib, uning asosiy o'zgaruvchisini yopiq deb

e'lon qilingan bo'lsa, u holda bu xossa voris sinfda ishlatilishi mumkin, lekin uning asosiy yopiq o'zgaruvchisiga bevosita kirish mumkin emas.

Quyida Asos va Balandlik o'zgaruvchilari xossalarga aylantirilgan Tekislik sinfining varianti keltirilgan.

// yopiq sinf a'zolarining qiymatlarini o'rnatish va olish uchun ochiq xossalardan foydalanish.

```
using System;
class Tekislik
{
    double a; // yopiq o'zgaruvchi
    double h; // yopiq o'zgaruvchi
    // Ikki o'lchovli obyektning asos va balandlik xossalari.
    public double Asos
    {
        get { return a; }
        set { a = value < 0 ? -value : value; }
    }
    public double Balandlik
    {
        get { return h; }
        set { h = value < 0 ? -value : value; }
    }
    public void Show()
    {
        Console.WriteLine("Asosva balandlik: " +
            Asos + " va " + Balandlik);
    }
}
// Tekislik sinfidan voris olingan Uchburchak sinfi
class Uchburchak : Tekislik
{
    public string UchburchakTuri; // Uchburchak turi
    // Uchburchak yuzasini qaytarish
    public double Yuza()
    {
        return Asos * Balandlik / 2;
    }
    // Uchburchak turini ko'rsatish
    public void ShowUchburchakTuri()
    {
        Console.WriteLine("Uchburchak: " + UchburchakTuri);
    }
}
class Shapes2
{
    static void Main()
    {
        Uchburchak t1 = new Uchburchak();
        Uchburchak t2 = new Uchburchak();
        t1.Asos = 4.0;
        t1.Balandlik = 4.0;
        t1.UchburchakTuri = "teng yonli";
    }
}
```

```

t2.Asos = 8.0;
t2.Balandlik = 12.0;
t2.UchburchakTuri = "to'g'ri burchakli";
Console.WriteLine("t1 obyekt bo'yicha: ");
t1.ShowUchburchakTuri();
t1.Show();
Console.WriteLine("Yuza: " + t1.Yuza());
Console.WriteLine();
Console.WriteLine("t2 obyekt bo'yicha: ");
t2.ShowUchburchakTuri();
t2.Show();
Console.WriteLine("Yuza: " + t2.Yuza());
}
}

```

Ushbu misolda Asos va Balandlik xossalari Tekislik sinfining a va h yopiq a'zolariga kira oladi, ular uchburchakning asosi va balandligini saqlaydi. Tekislik sinfining a va h a'zolarining qiymatlari yopiq bo'lsa ham, tegishli ochiq xossalardan foydalangan holda o'rnatilishi va olinishi mumkin.

Asosiy va voris sinflar ba'zan mos ravishda **super sinf** va **qism sinf** deb ham ataladi.

Sinf a'zolariga xavfsiz kirishni tashkil etish

Yuqorida aytib o'tilganidek, tayanch sinfning yopiq a'zosi voris olingan sinfga kira olmaydi. Voris olingan sinfdan tayanch sinfning ma'lum bir a'zosiga kirish uchun ushbu a'zo ochiq bo'lishi kerak. Ammo sinf a'zosini hammaga ochiq qilib qo'ysangiz, u barcha kodlar uchun ochiq bo'ladi, bu har doim ham effekt bermaydi.

Himoyalangan a'zo sinf iyerarxiyasida ochiq, lekin bu iyerarxiyadan tashqarida yopiqdir. Himoyalangan a'zo *protected* kirish modifikatori bilan yaratiladi. Agar sinf a'zosi *protected* deb e'lon qilinsa, u yopiq bo'lib qoladi, himoyalangan a'zo voris bo'lib qolgan hollar bundan mustasno.

Quyida *protected* kirish modifikatoridan foydalanishga oid misol keltirilgan.

```

// protected kirish modifikatoridan foydalanish
using System;
class B
{
    protected int i, j; // B sinfining yopiq a'zolari,
                        // lekin D sinfi uchun ochiq
    public void Set(int a, int b)
    {
        i = a;
        j = b;
    }
    public void Show()
    {
        Console.WriteLine(i + " " + j);
    }
}
class D : B
{
    int k; // yopiq a'zo
        // B sinfning i va j a'zolari D sinfi uchun ochiq
    public void Setk()
    {
        k = i * j;
    }
}

```

```

    }
    public void Showk()
    {
        Console.WriteLine(k);
    }
}
class ProtectedDemo
{
    static void Main()
    {
        D ob = new D();
        ob.Set(2, 3); // amal qiladi, chunki u D sinfi uchun ochiq
        ob.Show(); // amal qiladi, chunki u D sinfi uchun ochiq
        ob.Setk(); // amal qiladi, chunki u D sinf a'zosi
        ob.Showk(); // amal qiladi, chunki u D sinf a'zosi
    }
}

```

Ushbu misolda D sinf B sinfdan olingan voris sinf bo'lib, tayanch sinf a'zolari i va j *protected* deb e'lon qilinadi va shuning uchun Setk() metodi uchun mavjud. Agar B sinfining i va j a'zolari yopiq deb e'lon qilingan bo'lsa, u holda ular D sinfiga kira olmaydi va yuqoridagi kodni tuzib bo'lmaydi.

Xavfsiz kirish barcha holatlar uchun mos emas. Agar Asos va Balandlik a'zolari *public* bo'lsa, ularga berilgan qiymatlar nazorat qilinishi zarur, bunday holda, kirishni oldini olish o'rniga, kirishni boshqarish uchun xossalardan foydalanish yaxshiroq yechim bo'ladi. Agar butun sinf iyerarxiyasi uchun ochiq bo'lgan, ammo kodning qolgan qismi uchun yopiq bo'lishi kerak bo'lgan sinf a'zosini yaratish kerak bo'lsa, *protected* kirish modifikatoridan foydalanish zarur va sinf a'zosining qiymatiga kirishni nazorat qilish uchun xossadan foydalanish yaxshiroqdir.

Konstruktorlar va vorislik

Sinf iyerarxiyasi asosiy va voris sinflarga o'z konstruktorlariga ega bo'lish imkonini beradi. Quyidagi savol tug'ilishi tabiiy: voris sinf obyektini qurish uchun qaysi konstuktordan foydalaniladi: Tayanch sinf konstuktori, voris sinf konstuktori yoki ikkalasi? Bu savolga quyidagicha javob berish mumkin: tayanch sinf konstuktori obyektning asosiy qismini, voris sinfning konstuktori esa ushbu obyektning voris qismini tuzadi. Chunki voris bo'lgan sinfning har qanday elementlari noma'lum va tayanch sinfga kirish imkoni yo'q. Agar konstruktor faqat voris olingan sinfda aniqlangan bo'lsa, bu juda oddiy: olingan sinf objekti tuziladi va obyektning asosiy qismi avtomatik ravishda uning standart konstruktori tomonidan tuziladi. Misol:

```

// Uchburchak sinfiga konstruktor qo'shing.
using System;
class Tekislik
{
    double a;
    double h;
    // Obyektning asos va balandlik xossalari
    public double Asos
    {
        get { return a; }
        set { a = value < 0 ? -value : value; }
    }
    public double Balandlik
    {
        get { return h; }
        set { h = value < 0 ? -value : value; }
    }
}

```

```

    }
    public void Show()
    {
        Console.WriteLine("Asos va Balandlik: " +
            Asos + " va " + Balandlik);
    }
}
// Tekislik sinfidan voris olingan uchburchak sinf
class Uchburchak : Tekislik
{
    string UchburchakTuri;
    // Konstruktor
    public Uchburchak(string s, double w, double h)
    {
        Asos = w; // tayanch sinf a'zosini ishga tushirish
        Balandlik = h; // tayanch sinf a'zosini ishga tushirish
        UchburchakTuri = s; // tayanch sinf a'zosini ishga tushirish
    }
    // Uchburchak yuzasi
    public double Yuza()
    {
        return Asos * Balandlik / 2;
    }
    // Uchburchak turini chiqarish
    public void ShowUchburchakTuri()
    {
        Console.WriteLine("Uchburchak: " + UchburchakTuri);
    }
}
class Shapes3
{
    static void Main()
    {
        Uchburchak t1 = new Uchburchak("teng yonli", 4.0, 4.0);
        Uchburchak t2 = new Uchburchak("to'g'ri burchakli", 8.0, 12.0);
        Console.WriteLine("t1 obyekt bo'yicha: ");
        t1.ShowUchburchakTuri();
        t1.Show();
        Console.WriteLine("Yuza: " + t1.Yuza());
        Console.WriteLine();
        Console.WriteLine("t2 obyekt bo'yicha: ");
        t2.ShowUchburchakTuri();
        t2.Show();
        Console.WriteLine("Yuza: " + t2.Yuza());
    }
}

```

Bu misolda Uchburchak sinfi konstruktori Tekislik sinfining voris qilib olingan a'zolarini o'zining UchburchakTuri maydoni bilan birga ishga tushiradi.

Konstruktorlar ham tayanch sinfda, ham voris sinfda aniqlanganda, obyektни qurish jarayoni biroz murakkablashadi, chunki ikkala sinfning konstruktorlari bajarilishi kerak. Bunday holda, biz boshqa C# tilidagi kalit so'zga murojaat qilishimiz kerak: **base** - ikkita qo'llanmaga ega: birinchidan, tayanch sinf konstruktorini chaqirish uchun; ikkinchidan, voris olingan sinf a'zosining

yopiq tayanch sinf a'zosiga kirish uchun. *base* kalit so'zining ishlatilishi quyida muhokama qilinadi.

Tayanch sinf konstruktorlarini chaqirish

Voris olingan sinf konstruktorining kengaytirilgan shakli va *base* kalit so'zi bilan uning tayanch sinfida aniqlangan konstruktorni voris sinfda chaqirish mumkin. Quyida ularning kengaytirilgan umumiy shakli keltirilgan:

```
voris_sinf_konstruktori (parametr_ro'yxati): base (argument_ro'yxati)
{
    // konstruktor tanasi
}
```

Bu yerda, *parametr_ro'yxati* tayanch sinfdagi konstruktor tomonidan talab qilinadigan argumentlarni bildiradi.

base kalit so'zdan foydalanib, quyidagi dasturda Tekislik sinfining boshqa variantini ko'rib chiqamiz. Bu misol Asos va Balandlik xossalari ishga tushiruvchi konstruktorni belgilaydi. Keyinchalik bu konstruktorni Uchburchak sinfi konstruktori chaqiradi.

```
// Tekislik sinfiga konstruktor qo'shish
using System;
class Tekislik
{
    double a;
    double h;
    // Tekislik sinfi konstruktori
    public Tekislik(double w, double h)
    {
        Asos = w;
        Balandlik = h;
    }
    // Obyekt asosi va balandligi xossalari
    public double Asos
    {
        get { return a; }
        set { a = value < 0 ? -value : value; }
    }
    public double Balandlik
    {
        get { return h; }
        set { h = value < 0 ? -value : value; }
    }
    public void Show()
    {
        Console.WriteLine("Asos va Balandlik: " +
            Asos + " va " + Balandlik);
    }
}
// Tekislik sinfidan voris olingan Uchburchak sinfi
class Uchburchak : Tekislik
{
    string UchburchakTuri;
    // Tayanch sinf konstruktorini chaqirish
    public Uchburchak(string s, double w, double h) : base(w, h)
    {
        UchburchakTuri = s;
    }
}
```

```

    }
    // Uchburchak yuzasini hisoblash
    public double Yuza()
    {
        return Asos * Balandlik / 2;
    }
    // Uchburchak turini chiqarish
    public void ShowUchburchakTuri()
    {
        Console.WriteLine("Uchburchak: " + UchburchakTuri);
    }
}
class Shapes4
{
    static void Main()
    {
        Uchburchak t1 = new Uchburchak("teng yonli", 4.0, 4.0);
        Uchburchak t2 = new Uchburchak("to'g'ri burchakli", 8.0, 12.0);
        Console.WriteLine("t1 obyekt bo'yicha: ");
        t1.ShowUchburchakTuri();
        t1.Show();
        Console.WriteLine("Yuza: " + t1.Yuza());
        Console.WriteLine();
        Console.WriteLine("t2 obyekt bo'yicha: ");
        t2.ShowUchburchakTuri();
        t2.Show();
        Console.WriteLine("Yuza: " + t2.Yuza());
    }
}

```

Endi Uchburchak sinfi konstruktori quyidagicha e'lon qilinadi.

```
public Uchburchak(string s, double w, double h) : base(w, h)
```

Bunda Uchburchak() konstruktori w va h parametrlari bilan *base* metodni chaqiradi. O'z navbatida, Asos va Balandlik xossalarini w va h parametrlarining qiymatlari bilan ishga tushiradigan Tekislik() konstruktoriga chaqiradi.

base kalit so'zidan foydalanib, tayanch sinfda aniqlangan istalgan shakldagi konstruktorni chaqirish mumkin va faqat parametrlari o'tkazilgan argumentlarga mos keladigan konstruktor bajariladi. Misol tariqasida, quyida Tekislik va Uchburchak sinflarining to'liq versiyalari keltirilgan bo'lib, ularda standart konstruktorlar ham, bitta argumentni qabul qiluvchi konstruktorlar ham mavjud.

```

// Tekislik sinfiga qo'shimcha konstruktorlar qo'shish
using System;
class Tekislik
{
    double a;
    double h;
    // Standart konstruktor
    public Tekislik()
    {
        Asos = Balandlik = 0.0;
    }
    // Tekislik sinfining konstruktoriga
    public Tekislik(double w, double h)

```

```

{
    Asos = w;
    Balandlik = h;
}
// Bir xil asos va balandlikdagi konstruktorni qurish
public Tekislik(double x)
{
    Asos = Balandlik = x;
}
// Asos va Balandlik xossalari
public double Asos
{
    get { return a; }
    set { a = value < 0 ? -value : value; }
}
public double Balandlik
{
    get { return h; }
    set { h = value < 0 ? -value : value; }
}
public void Show()
{
    Console.WriteLine("Asos va Balandlik: " +
        Asos + " va " + Balandlik);
}
}
// Tekislik sinfidan voris olingan Uchburchak sinfi
class Uchburchak : Tekislik
{
    string UchburchakTuri;
    /* Standart konstruktor.
    Tekislik sinfida mavjud bo'lgan konstruktorni avtomatik ravishda chaqiradi*/
    public Uchburchak()
    {
        UchburchakTuri = "null";
    }
    // Uchta argumentni qabul qiluvchi konstruktor
    public Uchburchak(
        string s, double w, double h) : base(w, h)
    {
        UchburchakTuri = s;
    }
    // Teng yonli uchburchakni tuzish
    public Uchburchak(double x) : base(x)
    {
        UchburchakTuri = "teng yonli";
    }
    // Uchburchak yuzasini qaytarish
    public double Yuza()
    {
        return Asos * Balandlik / 2;
    }
}

```

```

// Uchburchak turini chiqarish
public void ShowUchburchakTuri()
{
    Console.WriteLine("Uchburchak: " + UchburchakTuri);
}
}
class Shapes5
{
    static void Main()
    {
        Uchburchak t1 = new Uchburchak();
        Uchburchak t2 = new Uchburchak("to'g'ri burchakli", 8.0, 12.0);
        Uchburchak t3 = new Uchburchak(4.0);
        t1 = t2;
        Console.WriteLine("t1 obyekt bo'yicha: ");
        t1.ShowUchburchakTuri();
        t1.Show();
        Console.WriteLine("Yuza: " + t1.Yuza());
        Console.WriteLine();
        Console.WriteLine("t2 obyekt bo'yicha: ");
        t2.ShowUchburchakTuri();
        t2.Show();
        Console.WriteLine("Yuza: " + t2.Yuza());
        Console.WriteLine();
        Console.WriteLine("t3 obyekt bo'yicha: ");
        t3.ShowUchburchakTuri();
        t3.Show();
        Console.WriteLine("Yuza: " + t3.Yuza());
        Console.WriteLine();
    }
}

```

base kalit so'zi voris olingan sinfda ko'rsatilganda, uning bevosita tayanch sinfidan konstruktor chaqiriladi. Shuning uchun *base* kalit so'zi har doim iyerarxiyadagi chaqiruvchi sinfdan yuqoridagi tayanch sinfga ishora qiladi. Bu hatto, ko'p darajali sinf iyerarxiyasi uchun ham amal qiladi. Argumentlar asosiy konstruktorga *base()* metodiga argumentlar sifatida uzatiladi. Agar kalit so'z bo'lmasa, tayanch sinfda ishlatiladigan standart konstruktor avtomatik ravishda chaqiriladi.

Vorislikda konstruktorni chaqirish tartibi

Konstruktorlarni chaqirishda quyidagi savollar tug'iladi: voris sinf obyekti qachon yaratiladi va qaysi konstruktor birinchi bo'lib bajariladi? Voris sinfda aniqlanganmi yoki tayanch sinfda aniqlanganmi? Agar A tayanch sinfi va voris olingan B sinf mavjud bo'lsa, A sinfning konstruktori B sinfning konstruktoridan oldin chaqiriladimi? Bu savollarga javob shundan iboratki, sinflar iyerarxiyasida konstruktorlar sinflar kelib chiqish tartibida chaqiriladi: asosdan vorisga. Bundan tashqari, bu tartib *base* kalit so'zdan qat'iy nazar bir xil bo'lib qoladi. Agar *base* kalit so'z ishlatilmasa, u holda standart konstruktor ya'ni parametrsiz konstruktor bajariladi. Quyidagi misolda konstruktorlarni chaqirish va bajarish tartibini ko'rsatadi.

```

using System;
// Tayanch sinf
class A
{
    public A()
    {

```

```

        Console.WriteLine("A sinfining tuzilishi");
    }
}
// A sinfdan voris olingan sinf yaratish
class B : A
{
    public B()
    {
        Console.WriteLine("B sinfining tuzilishi");
    }
}
// B sinfdan voris olingan sinf yaratish
class C : B
{
    public C()
    {
        Console.WriteLine("C sinfining tuzilishi");
    }
}
class Konstruktor
{
    static void Main()
    {
        C c = new C();
    }
}

```

Dasturni ishga tushirish natijasi.

A sinfining tuzilishi

B sinfining tuzilishi

C sinfining tuzilishi

Ko'rib turganingizdek, konstruktorlar o'z sinflaridan voris olingan tartibida chaqiriladi.